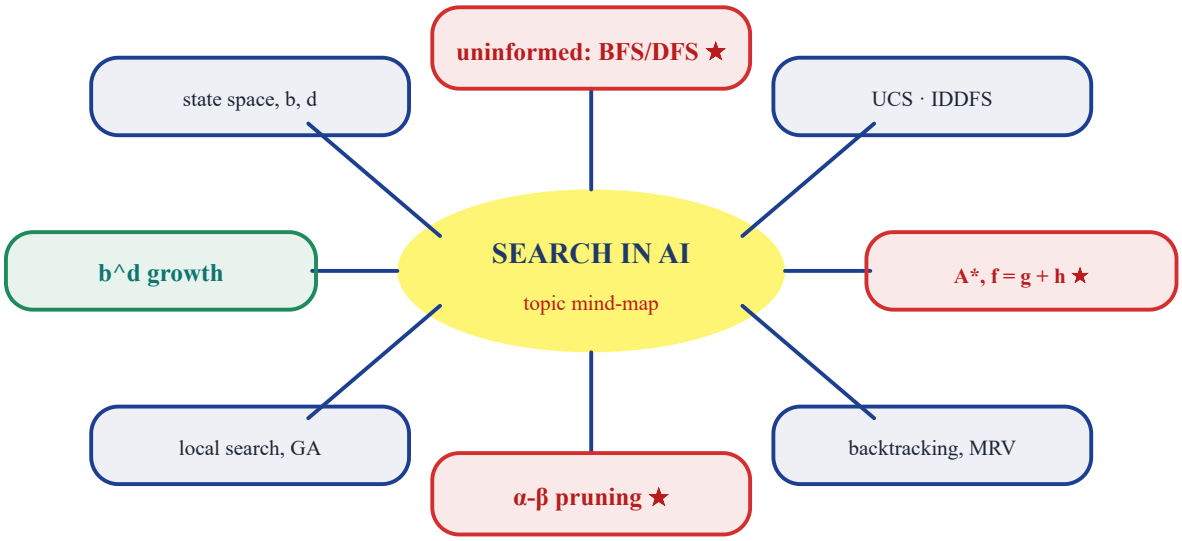
Search Techniques in AI

state space · BFS/DFS · UCS/IDDFS · greedy & A* (f = g + h) · minimax & α-β · backtracking: exam-ready in 12 sheets



↑ the whole topic on one page: red = high-yield, green = the key idea

Syllabus coverage

The Artificial Intelligence syllabus lists: "Introduction to old and new AI techniques; Knowledge representation; Propositional and first order logic; **Search techniques in AI**; Probabilistic reasoning; Natural language processing." This notebook covers exactly that line. Knowledge representation and logic have their own note; probabilistic reasoning and NLP follow this one.

High-yield list

① nodes = states, edges = actions
② BFS = queue, DFS = stack
③ UCS = priority queue on g
④ A*: f(n) = g(n) + h(n)
⑤ admissible: h(n) ≤ h*(n)
⑥ α-β best case O(b^(d/2))
⑦ CSP: MRV + forward check
⑧ complete ≠ optimal

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

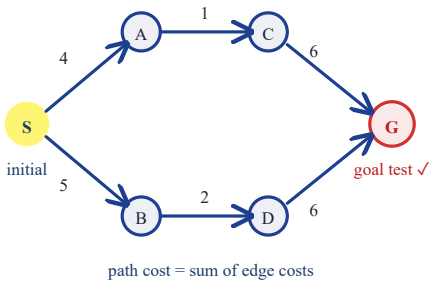
Contents

✓ Search as a state-space problem … p. 2
✓ Breadth-first & depth-first … p. 3
✓ UCS, depth-limited, IDDFS … p. 4
✓ Informed: greedy & A* … p. 5
✓ Admissible, consistent, heuristics … p. 6
✓ Local search, annealing, GA … p. 7
✓ Minimax & α-β pruning … p. 8
✓ Constraint satisfaction … p. 9
✓ Choosing a search, the cost of it … p. 10
✓ Night-before p. 11 · Practice p. 12

1 Search as a State-Space Problem

Definition: **State-space search** models a problem as a graph in which every **node is a complete state** of the world at one instant and every **edge is an action (operator)** that carries the agent to a successor state. The agent starts from the **initial state** and stops when a state passes the **goal test**.

> a **state** is a snapshot; a **node** is a snapshot + how we got there



path cost = sum of edge costs

every arrow is one move

The five parts of a formulation: **I** initial state · **A** actions · **T** transition results(s, a) = s′ · **G** goal test · **C** path cost $C(s_0, \ldots, s_k)$

Term	Meaning in one line
state space	every state reachable from the start
frontier	leaves generated, not yet expanded
closed list	nodes expanded — stops repeats
d, b	d = goal depth, b = mean successors
g(n)	cost of the best path start → n

Memory trick: "The **frontier** is the waiting room, the **closed list** is the memory." A node is born on the frontier when generated, joins the closed list when expanded, and dies once its children are waiting.

Exam trap: (i) A **state** is a complete description, not a history. (ii) Two nodes may hold the **same state** by different paths; only the cheaper should survive. (iii) Goal **test** ≠ goal **state**. (iv) b is an **average**, not the maximum.

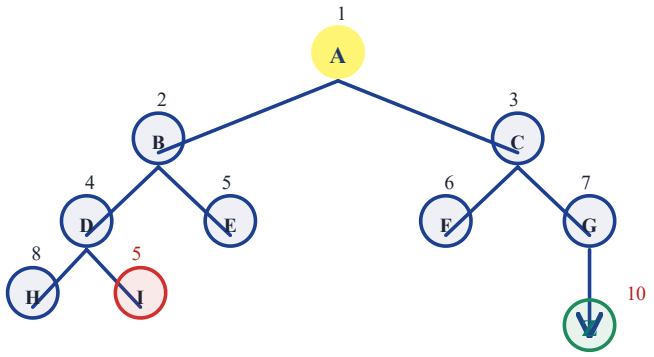
Worked example ✍ formulate the 8-puzzle: **State**: the 3 × 3 arrangement of tiles 1–8 plus the blank. **Operator**: slide a neighbouring tile into the blank — 2, 3 or 4 legal moves, so $b \leq 4$ and the effective branching factor is about **1.67**. **Goal test**: 1 2 3 / 4 5 6 / 7 8 blank. **Path cost**: number of moves. Only **9!/2 = 181 440** of the 9! arrangements are reachable, and the inversion test for reachability costs $O(n^2)$.

Tree search vs graph search: **Tree search** keeps no list, so it can revisit the same state for ever — completeness then needs finite b and finite d. **Graph search** keeps a closed list, so each state is expanded once, but it costs memory and may need **re-expansion**.

IT link: A program's "state" is a complete snapshot of memory and variables; in robotics the configuration space is the same idea. Any problem you can write as "state + action + goal test" is a search problem — and the algorithm simply walks that state graph.

2 Breadth-First & Depth-First Search

Definition: **Uninformed (blind) search** uses no knowledge of the problem beyond the initial state and the goal test — it is systematic, not clever. **BFS** expands the **shallowest unexpanded node**, so the frontier is a **FIFO queue**. **DFS** expands the **deepest unexpanded node**, so the frontier is a **LIFO stack** and recursion is natural.



blue = BFS order (FIFO queue), red = DFS order (LIFO stack); breadth walks the levels, depth meets the dead end at I

Property	Breadth-first	Depth-first
frontier	FIFO **queue**	LIFO **stack**
expands	shallowest left	deepest left
complete	Yes, if b, d finite	**No**
optimal	Yes, **unit cost**	No
time	O(b^d)	O(b^m)
space	**O(b^d)**	O(bm)
for	fewest steps	tight memory

Memory trick: "The **queue** is wide and blind — it remembers everything and knows nothing. The **stack** is narrow and brave — it remembers almost nothing and goes deep."

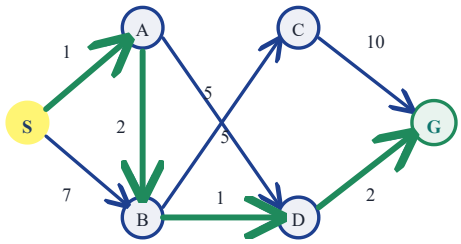
Exam trap: (i) "BFS finds the shortest path" means fewest **edges**; with unequal costs the fewest-edge path can be the **dearest** — use UCS. (ii) DFS is **not** complete and **not** optimal. (iii) BFS's real price is **space**. (iv) BFS = queue, DFS = stack; **every informed search uses a priority queue**.

Worked example ✍ count the expansions: In the tree drawn above the goal Z sits under G. **BFS** expands A, B, C, D, E, F, G and only then Z → **8 expansions**, holding a whole level in memory. **DFS** expands A, B, D, H, I (dead end), E, C, F, G, Z → **10 expansions**: it wanders into the dead ends H and I first. DFS keeps only the current path, O(bm), yet does **more** work — completeness has to be paid for.

IT link: BFS is the level-order walk behind "degrees of separation" in social graphs, BFS flood fill in a peer-to-peer network, and shortest-hop routing in a mesh. DFS is the DFS spanning tree behind cycle detection, topological sorting, connected components and the call stack of any recursive program.

3 Uniform-Cost, Depth-Limited & Iterative Deepening

Definition: **Uniform-cost search (UCS)** = BFS on costs: a priority queue ordered by the path cost g(n), so it expands the **cheapest node so far** — the uniform-cost algorithm of Dijkstra. **Depth-limited search (DLS)** = DFS cut off at depth **l**: complete iff l ≥ d, never optimal. **IDDFS** = DLS repeated with l = 0, 1, 2, … until the goal appears: **DFS memory, BFS completeness**.



green = UCS optimum, cost 6

cheapest node first — the rival route S-B-C-G costs 22

Step	Expanded	Open list after (node : g)
1	S (0)	A : 1 · B : 7
2	A (1)	B : **3** (was 7) · D : 6
3	B (3)	D : **4** (was 6) · C : 8
4	D (4)	G : 6 · C : 8
5	G (6)	goal: S-A-B-D-G

A node is **re-inserted** whenever a cheaper path to it is found — that rule is what makes UCS optimal instead of merely greedy.

Algorithm	Complete	Optimal	Time	Space
BFS	Yes	Yes, unit cost	O(b^d)	O(b^d)
DFS	**No**	**No**	O(b^m)	O(bm)
UCS (Dijkstra)	Yes	Yes	O(b^(C*/ε))	O(b^(C*/ε))
DLS	Yes if l ≥ d	No	O(b^l)	O(bl)
IDDFS	Yes	Yes, unit cost	O(b^d)	O(bd)

Memory trick: "UCS is **BFS with a metre tape** — same sweep, but it counts cost, not steps. IDDFS is **BFS wearing a DFS's memory**." Symbols: b = branching, d = goal depth, m = max depth, C* = optimal cost, ε = smallest step cost.

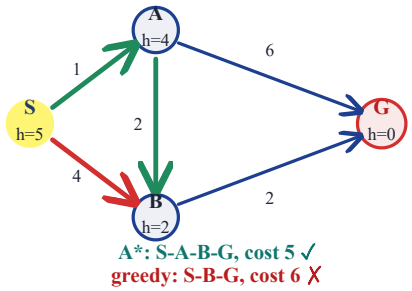
Exam trap: (i) UCS is optimal only because every step cost is **at least ε > 0**; a zero or negative cost edge breaks the argument. (ii) BFS is **not** optimal on weighted graphs — that is why UCS exists. (iii) IDDFS is optimal only for **unit** costs.

Worked example ✍ UCS on the graph drawn above: S→A = 1, S→B = 7, A→B = 2, A→D = 5, B→D = 1, B→C = 5, C→G = 10, D→G = 2. UCS expands S, A, B, D, G and returns S-A-B-D-G with cost 1 + 2 + 1 + 2 = **6**. The rival S-B-C-G costs 7 + 5 + 10 = **22** and is never opened. Watch: B's g falls from 7 to 3 when A is expanded.

Bidirectional search: Run a search **forward** from the start and **backward** from the goal at the same time and meet in the middle: a d-deep tree of branching b becomes two trees of depth d/2, so the work falls from **O(b^d)** to about **O(b^(d/2))**.

4 Informed Search: Greedy Best-First & A*

Definition: **Informed (heuristic) search** uses domain knowledge: a **heuristic h(n)** estimates the cost from n to the goal. All the best-first algorithms keep the frontier in a **priority queue** and pop the node with the lowest f, differing only in f: **UCS** f = g(n), **greedy** f = h(n), **A*** f = **g(n) + h(n)** — g(n) paid so far, h(n) still to come.



same graph, same heuristic — only f decides which path comes back

Algorithm	f(n)	Ranks by	Complete	Optimal
UCS	g(n)	cost so far	Yes	Yes
Greedy	h(n)	looks closest	No	No
A*	**g(n) + h(n)**	cheapest route	Yes	Yes, if h admiss.

A* properties. Time **O(b^d)**, space **O(b^d)** — worst case no better than BFS; the payoff is the **constant factor**. Put h = 0 and A* collapses to UCS.

Worked example ✍️ the same graph, two answers: Edges S→A = 1, S→B = 4, A→B = 2, A→G = 6, B→G = 2; h(S) = 5, h(A) = 4, h(B) = 2, h(G) = 0.
Greedy best-first (f = h): expand S → A (4), B (2); expand **B**; B generates G (0); expand **G** → returns S-B-G, cost 4 + 2 = **6**.
A* (f = g + h): expand S (f = 5) → A (g 1, f 5), B (g 4, f 6); expand **A** → B is re-reached with g = 3 < 4, so B becomes g 3, f 5; expand **B** → G is re-reached with g = 5 < 7; expand **G** → returns S-A-B-G, cost 1 + 2 + 2 = **5**.

Memory trick: "**g is the bill already paid, h is the doctor's guess, f is the total bill.**" Greedy looks only at the guess and throws the bill away; A* adds them — which is exactly why greedy can take the dear road.

Exam trap: (i) f = h(n) alone is **greedy**, f = g(n) alone is **UCS**, f = g·h is meaningless. (ii) A*'s optimality is a promise about h: an **over**estimating h silently returns a suboptimal path. (iii) Greedy best-first is **incomplete**. (iv) Tie-breaks change work, never the value.

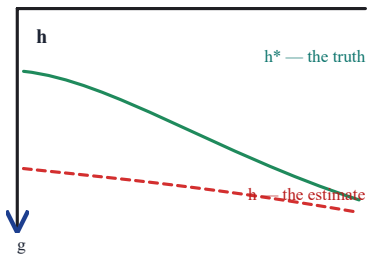
Where A* actually runs: Route finding (Google Maps, OSRM), robot path planning and typewriter word prediction. A* with a **consistent** heuristic is the dynamic-programming view of shortest paths: every node is settled once, in increasing order of f.

5 Admissible, Consistent & Good Heuristics

Definition: Write **h*(n)** for the true cost of the cheapest path from n to a goal. A heuristic is **admissible** if it **never overestimates**: h(n) ≤ h*(n) for every n. It is **consistent (monotone)** if **h(n) ≤ c(n, n′) + h(n′)** on every edge — the estimate may fall by at most the price of the move. Consistency implies admissibility.

Problem	Usual heuristic h(n)	Admissible?	Comment
route finding	straight-line distance	Yes	roads wind, so it is weak
8-puzzle	tiles out of place	Yes	very weak — ignores the blank
8-, 15-puzzle	**Manhattan distance**	Yes	row + column; standard
sliding puzzles	pattern databases	Yes	exact costs for a few tiles
8-queens	attacking pairs	No	divide by 2 to repair
travelling salesman	cheapest straight link	Yes	classic lower bound

Worked example 1 ✍ 8-puzzle, Manhattan: Start 2 8 3 / 1 6 4 / 7 5 _ , goal 1 2 3 / 4 5 6 / 7 8 _ . Tiles out of place: 1, 2, 4, 5, 6, 8 → **h = 6**. Manhattan: tile 1 → 1, tile 2 → 1, tile 4 → 2, tile 5 → 1, tile 6 → 1, tile 8 → 2 → **h = 8**. Each move shifts one tile one square, so h* ≥ 8.



an admissible estimate never rises above the true cost

Worked example 2 ✍ the admissibility audit: S→A = 2, S→B = 1, A→C = 2, B→C = 4, B→D = 5, C→G = 3, D→G = 2 with h(S) = 5, h(A) = 4, h(B) = 4, h(C) = 3, h(D) = 2, h(G) = 0. Backwards from the goal: h*(G) = 0, h*(C) = 3, h*(D) = 2, h*(A) = 5, h*(B) = 7, h*(S) = 7. Compare 5 ≤ 7, 4 ≤ 5, 4 ≤ 7, 3 ≤ 3, 2 ≤ 2, 0 ≤ 0 — **admissible**, and consistent too.

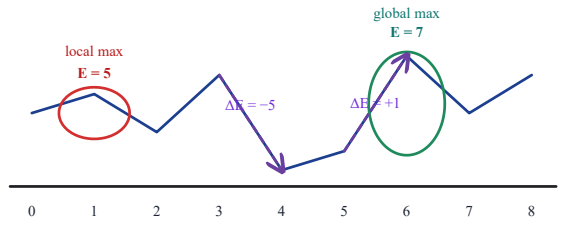
Memory trick: "**Admissible aims low.** Consistency is cautious: the estimate may fall by at most what the move costs." **Consistency ⇒ admissibility**, never the reverse.

Exam trap: (i) Admissible means **never overestimate** — h = 0 is trivially admissible and useless. (ii) h estimates what is **left**; h* is the truth. (iii) **Tree search** A* needs admissible h, **graph search** A* needs consistency. (iv) Manhattan distance is **not** the count of tiles out of place.

How good is "good"? ✍: If h = c · h* with a constant c > 0, A* behaves roughly like an **uninformed search with branching factor b^δ**, where δ = h* − h: on the 8-puzzle the tile-count heuristic is like b ≈ 3, Manhattan like b ≈ 1.4.

6 Local Search: Hill Climbing, Annealing, GA

Definition: **Local search** keeps only **one current state** — never a tree of them — and scores it with an **evaluation function E(s)** instead of a path cost. It suits spaces too large to build into a graph, and continuous ones. **Hill climbing** moves to a neighbour with better E and stops where none is better: a **local optimum**, which need not be the global one.



E(x) for x = 0…8; hill climbing stops at x = 1, annealing jumps the valley

From **x = 3** (E = 6) the neighbours x = 2 (3) and x = 4 (1) are both worse, so hill climbing stops at the **local maximum 6** while the global maximum 7 at x = 6 lies two steps away.

Where they are used. Hill climbing: VLSI layout, channel assignment. Annealing: 8-queens, tour improvement, chip placement. Genetic algorithms: antenna placement.

x	0	1	2	3	4	5	6	7	8
E(x)	4	**5**	3	6	1	2	**7**	4	6

Method	What it does	Local optimum?
Hill climbing	move to a better neighbour, stop when none is	No — stuck
Random restart	restart from a random state, keep the best	Yes, by luck
Simulated annealing	accept a worse move with p = **e^(−ΔE/T)**, T falling	Yes while T is high
Genetic algorithm	a **population**: selection, crossover, mutation	Yes, many live at once
Tabu search	forbid the last few moves (short memory)	Yes, it walks out

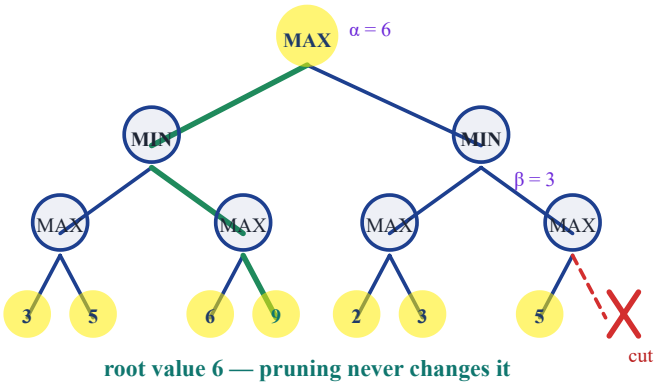
Worked example ✍️ annealing over E(x): At x = 3 (E = 6) the neighbours x = 2 (E = 3) and x = 4 (E = 1) are downhill by ΔE = −5 and −3. With T = 2 the move to x = 4 is accepted with probability e^(−5/2) ≈ **0.082** — unlikely, but not impossible. From x = 4 the move to x = 5 raises E to 2, so it is accepted outright; then x = 6 gives E = 7, the global maximum. As T → 0 the probability → 0 and annealing ends where hill climbing ends.

Memory trick: "**Annealing is eating a cold dinner** — to escape a bad meal you must first accept a worse one. A **genetic algorithm breeds a population**: fitness, selection, crossover, mutation."

Exam trap: (i) Hill climbing stuck at a local maximum is the **defining** failure, not a bug. (ii) The acceptance probability falls both as T falls and as the move gets worse. (iii) GA operators are **selection, crossover, mutation**. (iv) Local search keeps **no path**.

7 Adversarial Search: Minimax & α-β Pruning

Definition: When the states are shared by two opponents the graph is a game tree: one side wants the utility **maximised** (MAX), the other wants it **minimised** (MIN). **Minimax** walks the depth-first tree alternately, returning the best reply at every turn: v(s) = utility(s) on a terminal node, **max** of its successors at a MAX node, **min** at a MIN node. Branching b to depth d costs **O(b^d)** node evaluations.



after the left subtree gives α = 6, at G the first leaf 5 already beats β = 3, so the leaf 8 is cut

Node	Left leaf	Right leaf	Value	Note
D (MAX)	3	5	5	max(3, 5)
E (MAX)	6	9	9	max(6, 9)
B (MIN)	D = 5	E = 9	**6**	min → **α = 6** at the root
F (MAX)	2	3	3	max(2, 3) → **β = 3** at C
G (MAX)	5	8	—	5 ≥ β already, so the leaf 8 is cut
C (MIN)	F = 3	G = 5	3	min(3, 5)
root (MAX)	B = 6	C = 3	**6**	max(6, 3) = 6

Memory trick: **"α is MAX's best so far, β is MIN's best so far.** If a node's best can no longer beat what the opponent already has, cut it." Update α at MAX nodes only, β at MIN nodes only; α ≤ β always holds at a node.

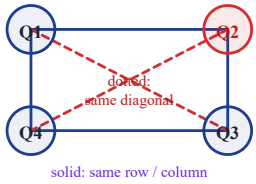
Exam trap: (i) Pruning changes the **work**, never the **value**. (ii) Move order is everything: best first gives **O(b^(d/2))**, random order about O(b^(3d/4)), worst order the full O(b^d). (iii) Minimax assumes **optimal play by both sides**; real opponents are not minimax.

Worked example ✍️ the saving in numbers: Full minimax evaluates 8 leaves; with α-β and the order drawn above it evaluates **7** — one leaf saved. The effect grows with depth: for b = 2 and d = 10 plain minimax needs 2^10 = **1024** leaves while perfect-ordering α-β needs about 2^5 = **32**, the same time but **twice the depth**. Every modern chess engine adds transposition tables, killer moves and move-ordering heuristics on top of α-β.

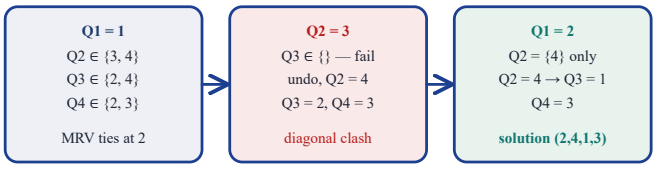
8 Constraint Satisfaction & Backtracking

Definition: A **constraint satisfaction problem** is the triple (X, D, C): a set of **variables X**, a **domain D** for each variable, and **constraints C** that must all hold at once. A **solution** is a complete assignment — there is no path, no cost and no intermediate reward, which is why we do **not** run BFS or A* here. The standard solver is **backtracking search**: assign a value, recurse, undo the assignment when a constraint fails.

Ingredient	Meaning
variable, domain	a thing to decide; its values
unary/binary/n-ary	constraint over 1, 2, k variables
constraint graph	variables as nodes, an edge per pair
backtracking	assign, recurse, **undo on failure**
forward checking	prune neighbours' domains now
MRV (fail-first)	variable with the **fewest values left**
LCV	value that constrains others **least**
arc consistency AC-3	every value has support next door
min-conflicts	local search on assignments



the 4-queens graph is complete — every pair of queens is constrained



forward checking after every assignment

Worked example ✍️ 4-queens: Q1 = 1 gives Q2 ∈ {3,4}, Q3 ∈ {2,4}, Q4 ∈ {2,3}: every domain has two values, so MRV may take any. Q2 = 3 empties Q3; Q2 = 4 forces Q3 = 2, which leaves Q4 = 3, a diagonal clash — so Q1 = 1 is dead. **Q1 = 2** leaves Q2 ∈ {4}: MRV forces **Q2 = 4**, Q3 is forced to 1 and Q4 to 3, giving **(2, 4, 1, 3)** in four assignments.

Memory trick: **"MRV asks the most constrained question first; LCV tries the least annoying answer first.** Forward checking crosses out the answers your neighbours just lost."

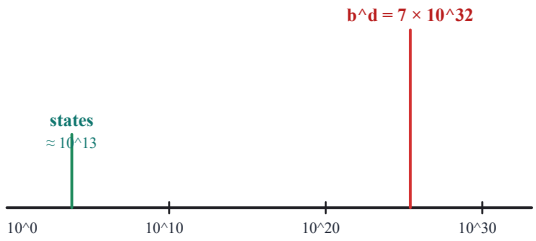
Exam trap: (i) A CSP is **not** a path search: only the final assignment matters. (ii) Forward checking is **not** arc consistency — it never looks past the neighbours. (iii) MRV chooses the **variable**, LCV the **value**. (iv) Backtracking is complete but exponential.

Applications: Sudoku, timetabling, rostering, cryptarithmetic, scheduling, register allocation in compilers. On an 8 × 8 board plain backtracking needs on the order of **10^5** nodes for the first solution; MRV plus forward checking brings that down to a few hundred.

9 Choosing a Search & What It Costs

Algorithm	Complete	Optimal	Time	Space	Frontier
BFS	Yes	Yes, unit	O(b^d)	O(b^d)	FIFO queue
DFS	No	No	O(b^m)	O(bm)	LIFO stack
UCS	Yes	Yes	O(b^(C*/ε))	O(b^(C*/ε))	queue on g
IDDFS	Yes	Yes, unit	O(b^d)	O(bd)	stack per depth
Greedy b-f	No	No	O(b^m)	O(b^m)	queue on h
A*	Yes	Yes, admiss. h	O(b^d)	O(b^d)	queue on g + h
α-β	Yes	Yes, same value	O(b^(d/2)) best	O(bd)	bounds α, β

If the problem looks like…	Use
fewest steps, all equal	BFS, or IDDFS when memory is tight
deep search, little memory	DFS, DLS, IDDFS
roads, money, fuel	UCS / Dijkstra
roads with a map at hand	A* with straight-line distance
a board game, an opponent	minimax + α-β
timetable, roster, Sudoku	backtracking + MRV + f.c.



the 15-puzzle: b = 2.13, d = 100 gives b^d ≈ 7 × 10^32 nodes, yet only 16!/2 ≈ 1.05 × 10^13 states exist

Exam trap: (i) **Complete ≠ optimal**: BFS and IDDFS are complete but optimal only for unit costs. (ii) A*'s O(b^d) is a **worst case**. (iii) α-β saves **time, never correctness**. (iv) A heuristic admissible for one problem is useless for another.

IT link: Dijkstra/UCS powers network routing tables; A* powers road routing and robot path planning; BFS powers level-order traversal and shortest hops; IDA* powers the standard 15-puzzle solver; MRV plus forward checking is what every Sudoku solver uses; α-β is in every chess and Go engine.

Worked example ✍️ why heuristics are not optional: For the 15-puzzle the tree an uninformed search must face has about **7 × 10^32** nodes at b = 2.13 and d = 100, although the whole problem contains only 16!/2 ≈ **1.05 × 10^13** distinct states. With an admissible Manhattan heuristic the effective branching factor collapses to roughly 1.4 — the difference between a search that finishes and one that never will.

Night-Before-the-Exam Sheet

Fact sheet:

node = state + parent + action + g(n)

b = branching, d = goal depth, m = max depth

UCS: priority queue on g(n)

A*: f(n) = g(n) + h(n)

Greedy: f(n) = h(n) alone

A* with h = 0 is UCS

admissible: $h(n) \le h^*(n)$

consistent: $h(n) \le c(n,n') + h(n')$

8-puzzle: 9!/2 = 181 440, b ≈ 1.67

Manhattan = row + column

α-β: O(b^(d/2)), random O(b^(3d/4))

CSP = (X, D, C); MRV = fewest values

annealing: accept $p = e^{(-\Delta E/T)}$

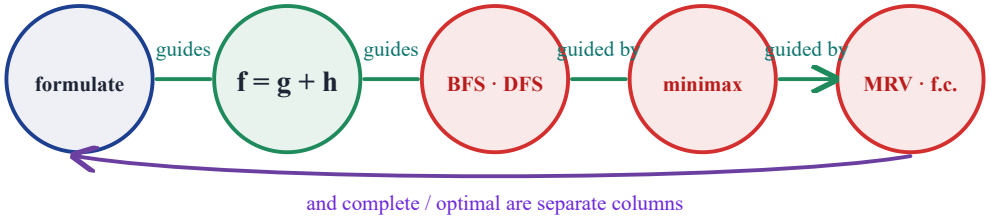
GA: selection, crossover, mutation

Top one-liners for MCQs

1. BFS frontier = FIFO **queue**; DFS = LIFO **stack**
2. BFS optimal only for **unit** step costs
3. DFS is neither complete nor optimal
4. UCS = Dijkstra, optimal, needs $\varepsilon > 0$
5. IDDFS = BFS completeness + DFS memory O(bd)
6. greedy best-first: f = h, not optimal
7. A* optimal ⟸ h admissible (never overestimates)
8. Consistency ⇒ admissibility
9. Manhattan distance is the 8-puzzle heuristic
10. Minimax: max at MAX nodes, min at MIN nodes
11. α-β prunes work, never the value
12. CSP = variables, domains, constraints — no path

Top traps: BFS optimal ≠ BFS always optimal (weighted graphs need UCS) · f = h is greedy, f = g is UCS, f = g + h is A* · h = 0 is admissible but useless · DFS needs a depth limit on infinite-depth graphs · hill climbing stops at a **local** optimum · α is updated at MAX nodes, β at MIN nodes · forward checking prunes domains, arc consistency is stronger.

All tricks together: "The frontier is the waiting room, the closed list is the memory" · "the queue is wide and blind, the stack is narrow and brave" · "UCS is BFS with a metre tape; IDDFS is BFS wearing a DFS's memory" · "g is the bill paid, h is the guess, f is the total" · "admissible aims low; consistency is cautious" · "annealing eats a cold dinner, a GA breeds a population".



one formulation, one priority rule, five algorithms — plus two columns that are not the same

✓ Practice MCQs (exam pattern, 10 marks)

1. BFS frontier is a:
(a) LIFO stack (b) priority queue
(c) search tree (d) FIFO queue

2. A* picks the next node:
(a) g(n) + h(n) (b) g(n) × h(n)
(c) h(n) alone (d) g(n) alone

3. h(n) is admissible if:
(a) overestimates (b) never exceeds
(c) constant time (d) falls by 1

4. α-β, perfect ordering:
(a) O(b × d) (b) O(d^b)
(c) O(b^(d/2)) (d) O(b^(2d))

5. IDDFS is preferred for:
(a) optimality (b) low memory
(c) no depth limit (d) no repeats

6. Plain DFS is complete if:
(a) costs are equal (b) graph is finite
(c) h is admissible (d) depth is finite

7. At a MIN node minimax:
(a) the minimum (b) the maximum
(c) the sum (d) α + β

8. MRV picks the variable:
(a) largest domain (b) most constraints
(c) fewest values (d) highest degree

9. Annealing may accept:
(a) worse moves (b) better moves
(c) only uphill (d) random moves

10. UCS optimal if costs ≥:
(a) ε > 0 (b) 1 exactly
(c) negative (d) zero

Answer key → 1-d · 2-a · 3-b · 4-c · 5-b · 6-d · 7-a · 8-c · 9-d · 10-a

Written question: Trace A* on the graph with edge costs S→A = 2, S→B = 1, A→C = 2, B→C = 4, B→D = 5, C→G = 3, D→G = 2 and heuristics 5, 4, 4, 3, 2, 0 for S, A, B, C, D, G. Give the expansion order, the path, its cost and an admissibility test. 10 marks

1. Use **f = g + h**, frontier in a priority queue on f. Start: open = {S (g = 0, f = 5)}. 1
2. Expand **S**: A (g = 2, f = 6), B (g = 1, f = 5); open = {B 5, A 6}. 1
3. Expand **B**: C (g = 5, f = 8), D (g = 6, f = 8); open = {A 6, C 8, D 8}. 1
4. Expand **A** (f = 6): C is reachable with g = 4 < 5, so C is **updated** to g = 4, f = 7; open = {C 7, D 8}. 1
5. Expand **C**: G (g 7, f 7); open = {G 7, D 8}; expand **G** — goal reached, order S, B, A, C, G. 2
6. Path **S-A-C-G**, cost 2 + 2 + 3 = **7**; S-B-C-G = 8 and S-B-D-G = 8 are dearer. 1
7. True remaining costs: h*(G) = 0, h*(C) = 3, h*(D) = 2, h*(A) = 5, h*(B) = 7, h*(S) = 7 — every h(n) ≤ h*(n), so h is **admissible** and consistent, and A*'s answer is guaranteed **optimal**. 3

Before the exam ✓

- ✓ Can write the five parts of a formulation
- ✓ Can quote the BFS/DFS complete + optimal columns
- ✓ Can trace A*, UCS and audit admissibility
- ✓ Can prune minimax and run MRV + forward checking